Updated 4h ago

Grouper Training - MCP AI - Lesson 06: Institutional documentation

By Chris Hyzer 7 min Add a reaction

Wiki Home	**Grouper Release Announcements**	**Grouper Guides**	**Grouper Deployment Guide**	**Community Contributions**

In this lesson you give the AI your institution's documentation. A daemon pulls wiki pages into the GTE as markdown, and the MCP `doc_search` tool indexes them, so answers about your own rules, owners, and processes come from your docs instead of guesses.

The GTE plays a fictional university, the Institute of Higher Education (IHE). Its simulated documentation is the four pages under Grouper Training Environment documentation, each with the Confluence label `gte-institutional-doc`. Glance at them before you start:

- Grouper folder structure and naming
- Reference groups and owners
- Access policy standards
- LDAP directory guide

Before you start: your GTE is running with its SSH tunnel open, and the Claude app is connected to `grouper-gte-local` as banderson, as in Grouper Training - MCP AI - Lesson 00: Pre-work.

Stay organized: name this lesson's conversations `Lesson 06: ...` and keep them in a sidebar section named `Lesson 06 Institutional docs`, as in Lesson 01.

Updated 4h ago

The daemon uses Playwright, which runs Chromium inside the Grouper container. On your GTE VM, open a root shell in the container:

```
./gte-shell
```

Install the operating system libraries Chromium needs (takes a few minutes):

```
. /usr/local/bin/librarySetupFilesForComponent.sh && setupFilesForC
```

Download Chromium as the tomcat user, and make the folder the daemon writes to:

```
su -s /bin/bash tomcat -c 'cd /home/tomcat && java -cp "/opt/groupe
mkdir -p /opt/grouper/docs/gte && chown tomcat:tomcat /opt/grouper/
```

The warning that your OS is not officially supported is expected. Type `exit` to leave the root shell.

Step 2: configure the daemon and the doc source

Download the attachment GteInstitutionalDocsHarvestDaemon.java . In the Grouper UI as banderson:

1. Go to "Miscellaneous" → "Configure" → "Configuration files" → "Import", choose `grouper.properties` , and import:

```
grouperGshTemplate.gteDocsHarvest.enabled = true
grouperGshTemplate.gteDocsHarvest.templateType = daemon
grouperGshTemplate.gteDocsHarvest.templateMode = compiled
grouperGshTemplate.gteDocsHarvest.templateName = GTE institutior
grouperGshTemplate.gteDocsHarvest.templateDescription = Pulls th
grouperGshTemplate.gteDocsHarvest.gshTemplate = //
grouper.mcp.docSearch.gteDocs.directory = /opt/grouper/docs/gte
grouper.mcp.docSearch.gteDocs.documentationForAiClient = Grouper
grouper.mcp.docSearch.gteDocs.reindexIntervalSeconds = 300
grouper.mcp.docSearch.gteDocs.boost = 2.0
```

2. Same import screen, choose `grouper-loader.properties` , and import:

```
otherJob.gteDocsHarvestDaemon.class = edu.internet2.middleware.gr
otherJob.gteDocsHarvestDaemon.scriptType = compiledJava
```

Updated 4h ago

3. Go to "Miscellaneous" → "GSH templates", edit `gteDocsHarvest`, and paste the whole Java file into the GSH script field. Save
4. Go to "Miscellaneous" → "Daemon jobs", and choose "Daemon actions" → "Schedule jobs", so Grouper picks up the new job

`documentationForAiClient` tells the AI what is in this source and when to search it. `boost` ranks it above the built-in Grouper wiki. `reindexIntervalSeconds` is how often Grouper re-reads the folder.

Step 3: run the daemon

1. Go to "Miscellaneous" → "Daemon jobs", filter for `gteDocsHarvest`, and choose "Job actions" → "Run job now"
2. After about 30 seconds, refresh. The status should be SUCCESS, and the job message should list four `wrote ...md` lines

Grouper builds the search index itself on the next doc_search after the files change (within `reindexIntervalSeconds`, here 5 minutes). No other job needs to run.

Step 4: ask about your institution

Use the copy button to copy this prompt into the Claude app in a new conversation:

```
At our institution, someone was locked by
security. Who can remove them from that group, and
which ticket queue do I use? Cite your sources.
```

Expected: only Security department staff (department 10902) can change ref:security:locked_by_ciso, use the **Security-Incident** queue, and the answer links to the reference groups page. These queue names exist only in the simulated docs, so this answer had to come from them.

If something does not work, copy and paste the error into Claude in the same conversation, and iterate until it works.

Updated 4h ago

The instructor goes over these, and the class does them if there is time.

Ask before the docs are loaded (do this before step 2)

Use the copy button to copy this prompt into the Claude app in a new conversation:

```
At our institution, someone was locked by
security. Who can remove them from that group, and
which ticket queue do I use? Cite your sources.
```

The AI can see the groups in Grouper, but it does not know your owners or ticket queues yet, so expect a general answer.

Check the run with AI

This needs "Admin read-only operations" access. If it is refused, check the run in the UI instead (step 3).

Use the copy button to copy this prompt into the Claude app in a new conversation:

```
Show me the most recent run of the
OTHER_JOB_gteDocsHarvestDaemon daemon and its job
message.
```

You should see SUCCESS with 4 pages written.

Look at the markdown

To look at the markdown, on the GTE VM:

```
./gte-shell
cd /opt/grouper/docs/gte
ls -l
cat 330203139.md
```

More questions

Use the copy button to copy this prompt into the Claude app in a new conversation:

Updated 4h ago

```
3  and what should my ticket include?
```

Expected: a ticket in the **IAM-Grouper** queue with the application name and owner, who should have access, and the target system. IAM creates it in 3 business days.

Use the copy button to copy this prompt into the Claude app in a new conversation:

```
1  Can I provision ref:student:students directly to
2  my application? What does our policy say, and how
3  long can a manual exception last?
```

Expected: no, build an access policy in the application folder and provision that. Manual exceptions last at most 1 year and are attested every 90 days.

Docs steer the work

The docs tell the AI how IHE builds access policies, and the MCP tools do it. This needs read-write access, so log in to MCP again and grant it. Do this in Claude Code on the command line. In a terminal, start Claude Code in the training folder:

```
1  cd ~/grouperMcpTraining
2  claude
```

1. Type `/mcp`, select "grouper-gte-local", then select "Re-authenticate"
2. Log in as banderson / password
3. On the consent page, check "Read-only operations" and "Read-write operations"
4. Under "Limit read-write scope", in "Folders", enter `app` and click "Add folder". The AI can then change groups only in the `app` folder
5. Approve
6. Type `/exit`, then quit the Claude app completely and open it again, so it uses the new login
7. Start a new conversation in the Claude app

Updated 4h ago

```
Following our institutional access policy
standards, set up a new application called wiki in
Grouper: create the folders and the allow, deny,
and policy groups the way our standards say. Show
me the plan first.
```

The plan should follow the access policy standards page: `wiki_allow`, `wiki_deny` (which includes ref:iam:global_deny), and the `wiki` policy group as a composite.

On your own: have AI build all of this

> Advanced, for after the training. There is no time to troubleshoot this in class. Expect several rounds of run, read the error, fix. You need MCP with "Read-only operations", "Admin read-only operations", and "Admin read-write operations" (to run daemons), and a root shell in the container (`./gte-shell`) for the setup commands the AI gives you.

Instead of using the provided daemon, ask AI to design and write one, then run it and fix it until it works. The Grouper source in `~/grouperMcpTraining/grouper` has the compiled Java daemon contract, the doc_search config, and the Grouper wiki (in `grouper-misc/grouper-docs/wikiMirror`).

1. Write the daemon

Use the copy button to copy this prompt into the Claude app in a new conversation:

```
Using the Grouper source in this folder, write a
compiled Java Grouper daemon
(GrouperTemplateDaemon) that uses Playwright to
harvest Confluence pages into markdown for the MCP
doc_search tool:
- Find the pages with the label
gte-institutional-doc on
https://docs.grouper.internet2.edu/wiki with a CQL
search through the Confluence REST API (anonymous,
```

Updated 4h ago

```
confluence to render the body (wiki content
.ak-renderer-document)
- Remove the Navigation include at the top of each
page (it renders as a nested
.ak-renderer-document), and convert the body HTML
to markdown with Flexmark
- Write one .md file per page with title and url
frontmatter to /opt/grouper/docs/gte, and delete
files for pages that lost the label, but do not
delete anything if the search finds no pages
- The daemon runs inside Grouper's Tomcat, which
does not see the tomcat user's home folder, so
pass PLAYWRIGHT_BROWSERS_PATH and
PLAYWRIGHT_SKIP_BROWSER_DOWNLOAD=1 to
Playwright.create with CreateOptions.setEnv,
pointing at where the browser will be installed
- Put each page's result and any error on the
loader log job message
Playwright and Flexmark jars are already in the
Grouper container.  Use the template config id
gteDocsHarvest and the daemon
otherJob.gteDocsHarvestDaemon.  Give me the config
for grouper.properties and
grouper-loader.properties as separate blocks,
including the grouper.mcp.docSearch config for
that folder.
```

The prompt includes what we learned building the provided daemon: where Confluence puts the page body, the Navigation include, and that Tomcat needs to be told where the browser is.

2. Check the container, and set it up if needed

Use the copy button to copy this prompt into the Claude app in a new conversation:

```
How do I check whether my GTE container is ready
to run that Playwright daemon inside Grouper's
Tomcat, and set up whatever is missing?  I get a
root shell in the container with ./gte-shell.  For
each of these, give me a command to check it and a
command to fix it if the check fails:
- The OS libraries headless Chromium needs (the
```

Updated 4h ago

```
11  /usr/local/bin/librarySetupFilesForComponent.sh)
12  - Chromium downloaded for the tomcat user, to the
13  same browsers path the daemon passes to Playwright
14  - The tomcat user can read that browsers path and
15  write to /opt/grouper/docs/gte
16  Then give me one command that checks all of them
17  at once.
```

Run the check command again after each fix, until everything passes.

3. Load it and run it

Use the copy button to copy this prompt into the Claude app in a new conversation:

```
1  I imported the config and pasted the Java into the
2  gteDocsHarvest GSH template.  Run the
3  OTHER_JOB_gteDocsHarvestDaemon daemon, then check
4  its result and job message.  If it failed, tell me
5  why and give me the fixed code or config.
```

Repeat this prompt after each fix until the daemon succeeds.

4. Check the docs are searchable

Use the copy button to copy this prompt into the Claude app in a new conversation:

```
1  Which documentation sources can doc_search use,
2  and what documents are in the gteDocs source?
3  Then answer: at our institution, who can remove
4  someone from the security lock group, and which
5  ticket queue do I use?  Cite your sources.
```

5. Extend it

Use the copy button to copy this prompt into the Claude app in a new conversation:

```
1  Change the daemon to also record when each page
2  was last updated in Confluence, and to skip pages
```

Updated 4h ago

```
6  deployment where the daemon and the MCP server run
7  in different containers.
```

Add a comment

Add a reaction